 # Experiment 6:

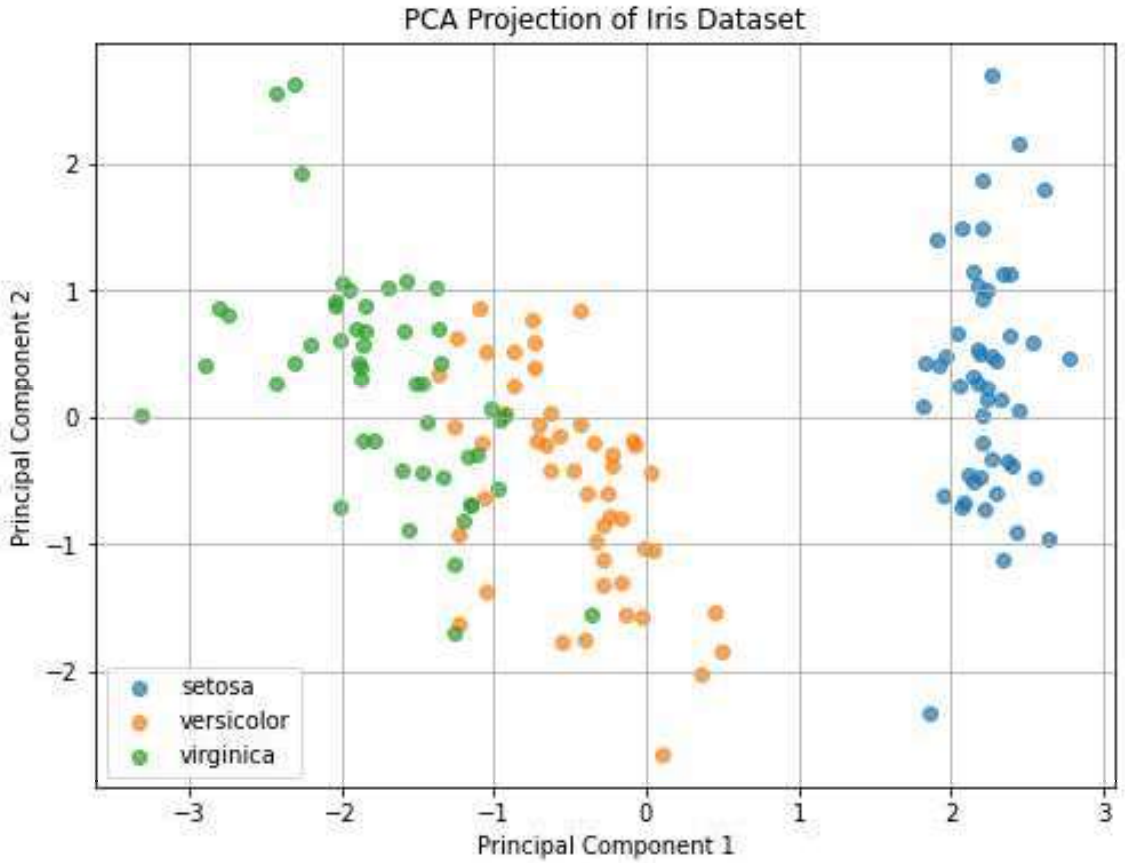
```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = load_iris()
X = iris.data  # Features
y = iris.target  # Class labels

# Standardize the dataset (mean = 0, variance = 1)
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Compute covariance matrix
cov_matrix = np.cov(X_scaled.T)

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors = np.linalg.eigh(cov_matrix)

# Sort eigenvalues and corresponding eigenvectors in descending order
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]

print("Eigen values=", eigenvalues)
# Choose number of principal components (m)
m = 2
principal_components = eigenvectors[:, :m].T
```

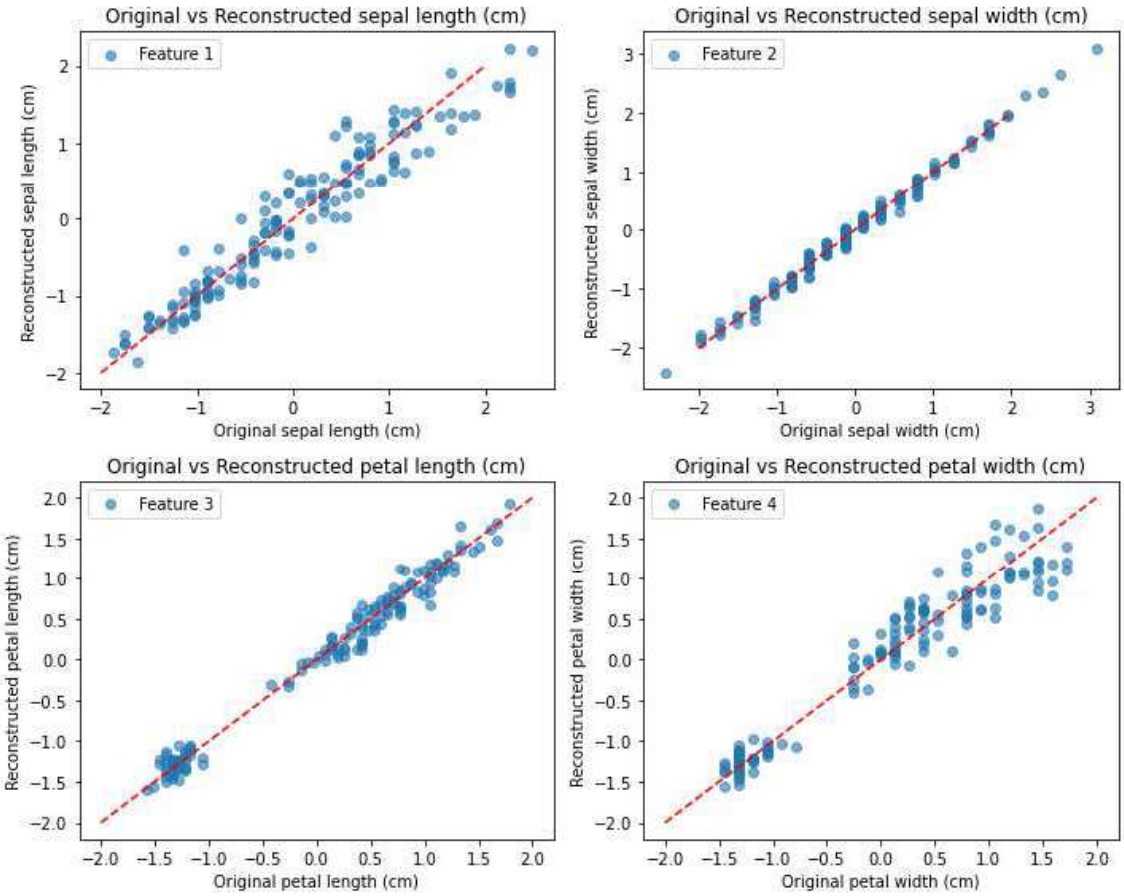
```
# Project data onto the first m principal components
X_projected = np.dot(principal_components, X_scaled.T).T

# Scatter plot of the projected data
plt.figure(figsize=(8, 6))
colors = ['r', 'g', 'b']
labels = iris.target_names
for i in range(3):
    plt.scatter(X_projected[y == i, 0], X_projected[y == i, 1], label=labels[i], alpha=0.7)

plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA Projection of Iris Dataset')
plt.legend()
plt.grid()
plt.show()

# Reconstruct the data from PC1 and PC2
X_reconstructed = np.dot(X_projected, principal_components)

# Scatter plot comparing original vs reconstructed features
feature_names = iris.feature_names
plt.figure(figsize=(10, 8))
for i in range(X.shape[1]):
    plt.subplot(2, 2, i + 1)
    plt.scatter(X_scaled[:, i], X_reconstructed[:, i], alpha=0.6, label=f'Feature {i+1}')
    plt.plot([-2, 2], [-2, 2], 'r--')  # Reference line (perfect reconstruction)
    plt.xlabel(f'Original {feature_names[i]}')
    plt.ylabel(f'Reconstructed {feature_names[i]}')
    plt.title(f'Original vs Reconstructed {feature_names[i]}')
    plt.legend()
```

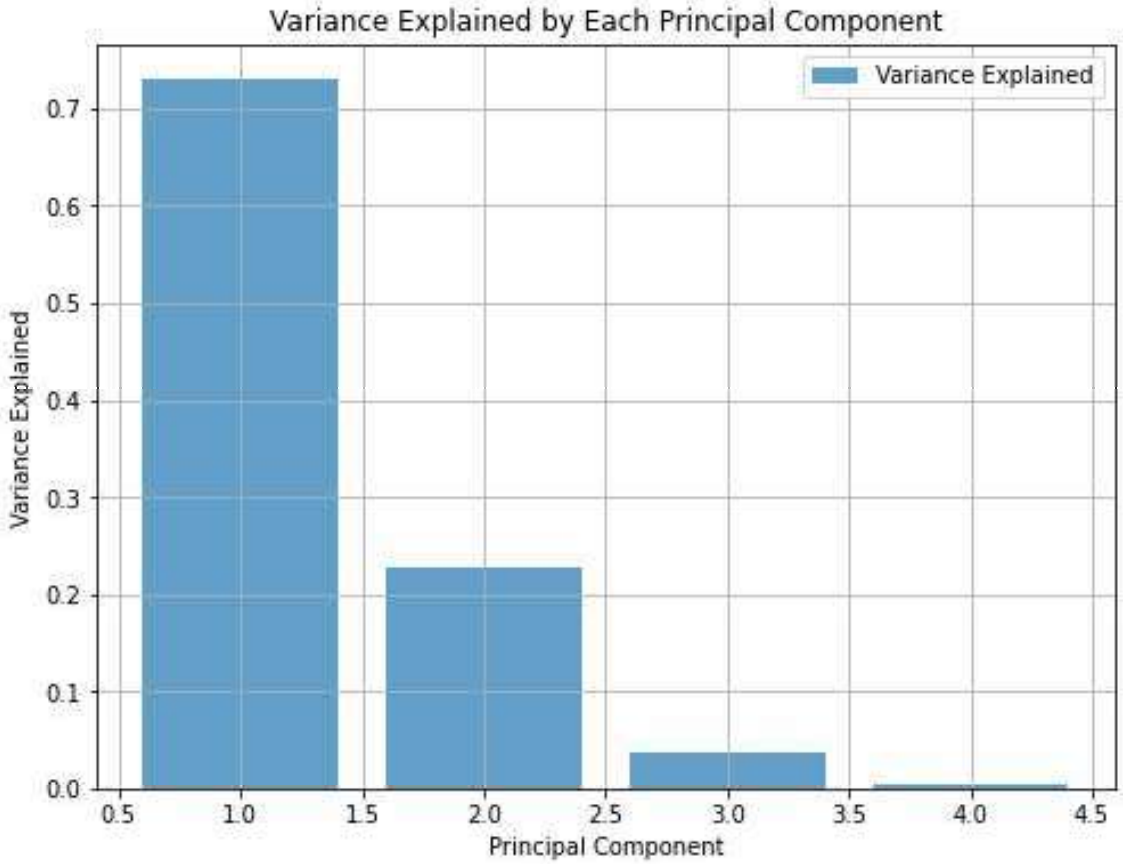
```
plt.tight_layout()

plt.show()


# Calculate explained variance

variance_explained = eigenvalues / np.sum(eigenvalues)

cumulative_variance_explained = np.cumsum(variance_explained)


# Plot variance explained

plt.figure(figsize=(8, 6))

plt.bar(range(1, len(variance_explained) + 1), variance_explained, alpha=0.7, label='Variance Explained')

plt.xlabel('Principal Component')

plt.ylabel('Variance Explained')

plt.title('Variance Explained by Each Principal Component')

plt.legend()

plt.grid()

plt.show()


# Plot cumulative variance explained

plt.figure(figsize=(8, 6))

plt.plot(range(1, len(cumulative_variance_explained) + 1), cumulative_variance_explained, marker='o', linestyle='-', color='r')

plt.xlabel('Number of Principal Components')

plt.ylabel('Cumulative Variance Explained')

plt.title('Cumulative Variance Explained by PCA')

plt.grid()

plt.show()


# Find minimum dimensions to retain 98% variance

min_dim_98pc = np.argmax(cumulative_variance_explained >= 0.98) + 1

print(f"Minimum dimensions needed to retain 98% variance: {min_dim_98pc}")
```

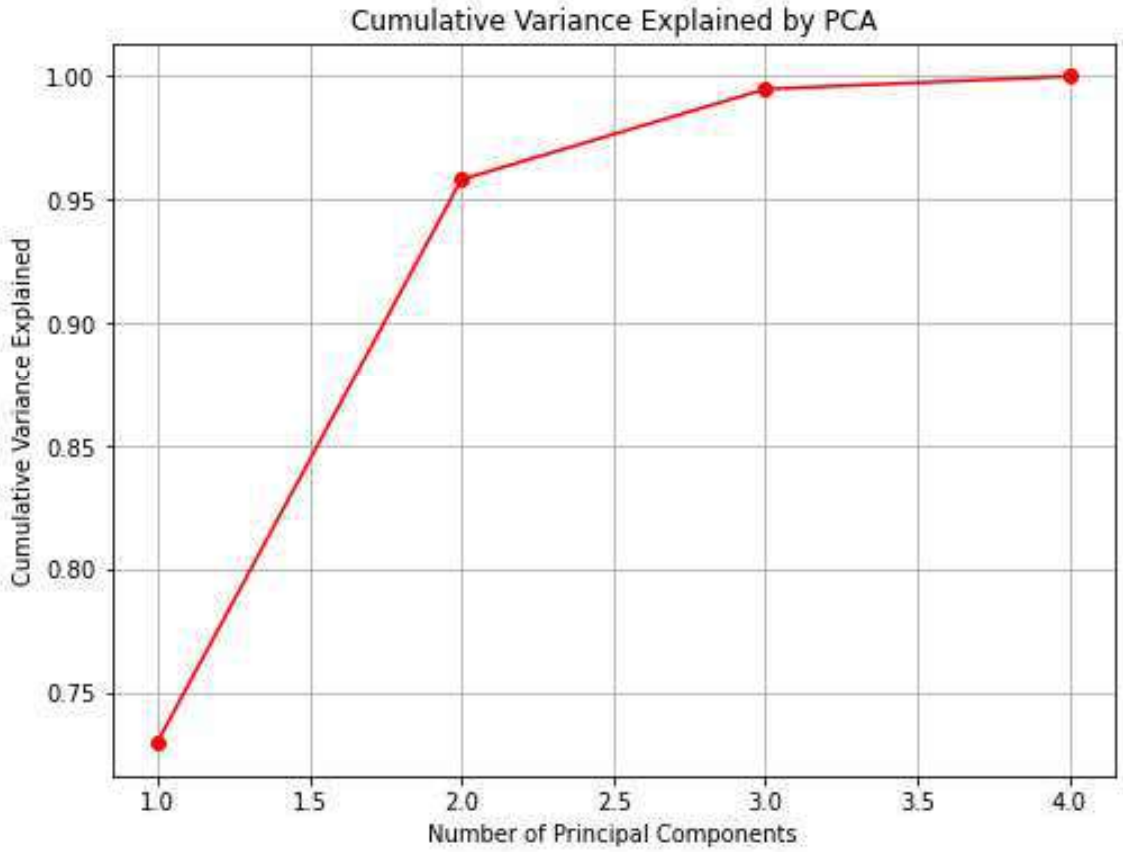
Results:

Eigen values= [2.93808505 0.9201649 0.14774182 0.02085386]




Original vs Reconstructed sepal length (cm)
Feature 1
Reconstructed sepal length (cm)
Original sepal length (cm)
Original vs Reconstructed sepal width (cm)
Feature 2
Reconstructed sepal width (cm)
Original sepal width (cm)
Original vs Reconstructed petal length (cm)
Feature 3
Reconstructed petal length (cm)
Original petal length (cm)
Original vs Reconstructed petal width (cm)
Feature 4
Reconstructed petal width (cm)
Original petal width (cm)


Variance Explained by Each Principal Component
Variance Explained
Variance Explained
Principal Component
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
4.5



Minimum dimensions needed to retain 98% variance: 3

 # Experiment 7:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.preprocessing import StandardScaler

# Load Iris dataset
iris = datasets.load_iris()
X, y = iris.data, iris.target

# Standardize the dataset (mean = 0, variance = 1)
scaler = StandardScaler()
X = scaler.fit_transform(X)
```

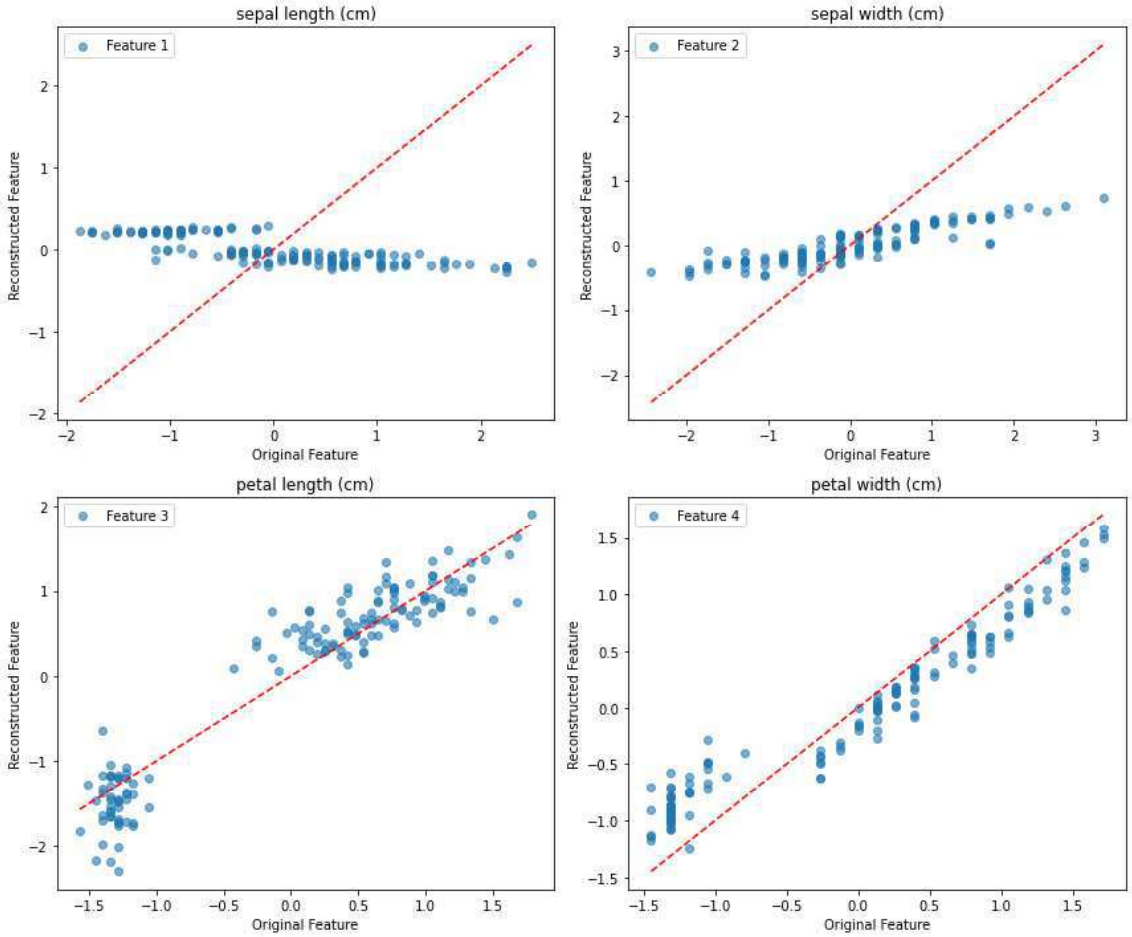
```
# Step 2: Compute Class Means
class_means = []
for label in np.unique(y):
    class_means.append(np.mean(X[y == label], axis=0))

# Step 3: Compute Within-Class Scatter Matrix
within_class_scatter = np.zeros((X.shape[1], X.shape[1]))
for label, mean_vec in zip(np.unique(y), class_means):
    class_scatter = np.cov((X[y == label] - mean_vec).T)
    within_class_scatter += class_scatter

# Step 4: Compute Between-Class Scatter Matrix
overall_mean = np.mean(X, axis=0)
between_class_scatter = np.zeros((X.shape[1], X.shape[1]))
for label, mean_vec in zip(np.unique(y), class_means):
    n = X[y == label].shape[0]
    mean_diff = (np.array(mean_vec) - np.array(overall_mean)).reshape(-1, 1)
    between_class_scatter += n * np.dot(mean_diff, mean_diff.T)

# Add regularization to within-class scatter matrix
lambda_ = 0.01  # Regularization parameter
within_class_scatter_reg = within_class_scatter + lambda_ *
np.identity(within_class_scatter.shape[0])

# Compute eigenvalues and eigenvectors
eigenvalues, eigenvectors =
np.linalg.eig(np.linalg.inv(within_class_scatter_reg).dot(between_class_scatter))

# Ensure real-valued outputs
eigenvalues = np.real(eigenvalues)
eigenvectors = np.real(eigenvectors)
```

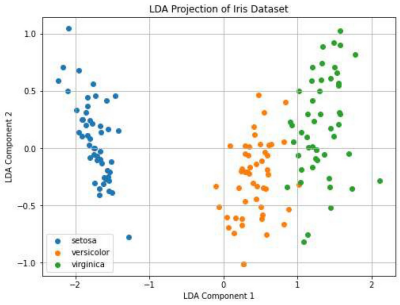
```
# Select discriminant directions
sorted_indices = np.argsort(eigenvalues)[::-1]
eigenvalues = eigenvalues[sorted_indices]
eigenvectors = eigenvectors[:, sorted_indices]

print("Eigenvalues=", eigenvalues)

# Choose number of discriminant directions, up to k-1, i.e., 2
m = 2
principal_components = eigenvectors[:, :m].T
X_projected = np.dot(principal_components, X.T).T

# Reconstruct features from LDA projection
X_reconstructed = np.dot(principal_components.T, X_projected.T).T

# Plot original vs. reconstructed features
feature_names = iris.feature_names
plt.figure(figsize=(12, 10))
for i in range(X.shape[1]):
    plt.subplot(2, 2, i + 1)
    plt.scatter(X[:, i], X_reconstructed[:, i], alpha=0.6, label=f'Feature {i+1}')
    plt.plot([min(X[:, i]), max(X[:, i])], [min(X[:, i]), max(X[:, i])], 'r--')  # Diagonal reference line
    plt.xlabel('Original Feature')
    plt.ylabel('Reconstructed Feature')
    plt.title(feature_names[i])
    plt.legend()
plt.tight_layout()
plt.show()

# Plot LDA projection
```

```
plt.figure(figsize=(8, 6))
for label in np.unique(y):
    plt.scatter(X_projected[y == label, 0], X_projected[y == label, 1], label=iris.target_names[label])
plt.xlabel('LDA Component 1')
plt.ylabel('LDA Component 2')
plt.legend()
plt.title('LDA Projection of Iris Dataset')
plt.grid()
plt.show()
```

Results:

Eigenvalues= [1.47976864e+03 1.36391897e+01 2.86826511e-14 2.86826511e-14]




LDA Projection of Iris Dataset
LDA Component 2
LDA Component 1
setosa
versicolor
virginica
1.0
0.5
0.0
−0.5
−1.0
−2
−1
0
1
2